

Day 6 — How Vanilla RAG, LlamaIndex, LangChain, LangGraph, and MCP Fit Together

We will use one fictional Disney-like system throughout:

| **MagicGuide** is an AI assistant for park guests and support agents.

Example user request:

| “Can my daughter ride Starlight Mountain, what is the current waiting time, and can you move our lunch reservation to 2:30 PM?”

This request needs three different capabilities:

- 1. Read trusted attraction rules.
- 2. obtain live operational information.
- 3. Change a reservation safely.

That is why a real production system may need retrieval, application components, workflow control, and external-tool connectivity.

1. Core integrated summary

The most important point is:

| **These five things are not five competing versions of the same product. They belong to different architectural categories.**

A simple mental model is:

Topic	Simple meaning
Vanilla RAG	A recipe for answering from external knowledge
LlamaIndex	A toolkit focused heavily on preparing, indexing, retrieving, and using enterprise data
LangChain	A toolkit for connecting models, prompts, tools, retrievers, and application components
LangGraph	A runtime for controlling stateful, multi-step AI workflows
MCP	A standard communication protocol for connecting AI applications to external tools and data

Think of building a restaurant:

- **Vanilla RAG** is the recipe.
- **LlamaIndex** manages the ingredients and food preparation.
- **LangChain** gives you reusable kitchen equipment.
- **LangGraph** is the kitchen manager controlling the order of work.
- **MCP** is the standard interface used to communicate with outside suppliers.

You do **not** need all five for every application.

A simple document chatbot may need only:

API + custom RAG + model + vector database

A complex Disney-like operational assistant may use:

```
FastAPI
  ↓
LangGraph workflow
├── LlamaIndex retrieval service
├── LangChain model and tool integrations
├── MCP clients
│   ├── Reservation MCP server
│   ├── Live operations MCP server
│   └── Guest-support MCP server
```

RAG combines model knowledge with retrieved external knowledge. The original RAG work described this as combining a model's internal, parametric memory with an external, non-parametric memory such as a searchable document index. ([arXiv](#))

The current official positioning of these frameworks also shows why their boundaries can look blurry: LlamaIndex supports agents, workflows, retrieval, and context augmentation; LangChain supports models, tools, agents, integrations, and retrieval; LangGraph focuses on low-level orchestration; and MCP standardizes connectivity. ([Developer Documentation](#))

2. Foundational category map

2.1 What is a retrieval pattern?

A **pattern** is a general solution shape.

It tells you:

- what major steps should happen;
- why those steps exist;
- how information should flow.

It does not force you to use a particular library.

Vanilla RAG is a pattern such as:

```
User question
  ↓
Search external knowledge
  ↓
Retrieve relevant passages
  ↓
Put passages into the model prompt
  ↓
Generate a grounded answer
```

You can implement this using:

- plain Python;
- LlamaIndex;
- LangChain;
- your own internal framework;
- cloud-managed retrieval services.

Therefore:

| RAG is not automatically LlamaIndex, LangChain, or LangGraph.

They are possible implementations around the RAG pattern.

2.2 What is a framework?

A **framework** is reusable software that provides:

- abstractions;
- components;
- interfaces;
- integrations;
- conventions;
- utility code.

Instead of writing every integration yourself, you use framework components.

For example:

```
documents = loader.load()
chunks = splitter.split_documents(documents)
retriever = vector_store.as_retriever()
response = model.invoke(prompt)
```

The framework may provide abstractions for:

- models;
- documents;
- embeddings;
- vector stores;
- tools;
- retrievers;
- agents;
- prompts.

Both LlamaIndex and LangChain are frameworks, although their centres of gravity differ.

2.3 What is an orchestration runtime?

An **orchestration runtime** controls how a process executes over time.

It answers questions such as:

- Which step runs first?
- Which step runs next?
- What happens when a step fails?
- Should we retry?
- Should we ask a human?
- How do we save the current state?
- Can we resume after interruption?
- Which steps may run in parallel?

LangGraph is primarily this layer.

Its central concepts are:

State	= current workflow information
Nodes	= work to perform
Edges	= what runs next
Runtime	= executes and persists the graph

The official LangGraph documentation describes it as a low-level orchestration runtime focused on durable execution, streaming, persistence, and human-in-the-loop control. It can mix predictable code steps with flexible model-driven steps. ([Docs by LangChain](#))

2.4 What is a protocol?

A **protocol** is an agreed communication contract.

It defines things such as:

- how two systems introduce themselves;
- how capabilities are discovered;
- how requests are formatted;
- how responses and errors are represented;
- how authentication may work;
- how connections begin and end.

A protocol does not normally decide your business workflow.

For example, HTTP defines how applications communicate over the web. HTTP does not decide whether your reservation should be cancelled.

Similarly:

MCP defines how an AI application communicates with MCP servers. It does not decide the complete business workflow.

MCP uses a host-client-server model and standardized concepts such as tools, resources, and prompts. Its protocol messages use JSON-RPC. ([Model Context Protocol](#))

2.5 Why people confuse these categories

The confusion happens because modern frameworks keep expanding.

For example:

- LlamaIndex can perform RAG, agents, and workflows.
- LangChain can perform RAG, tool calling, and agents.
- LangGraph can contain retrieval steps and tools.
- MCP resources can provide data used by RAG.
- MCP tools can appear as tools inside LangChain or LangGraph.
- LangChain's higher-level agent API uses LangGraph as its underlying runtime. ([Docs by LangChain](#))

Therefore, the products overlap in **capability**, but they differ in **primary responsibility**.

A useful Staff Engineer question is not:

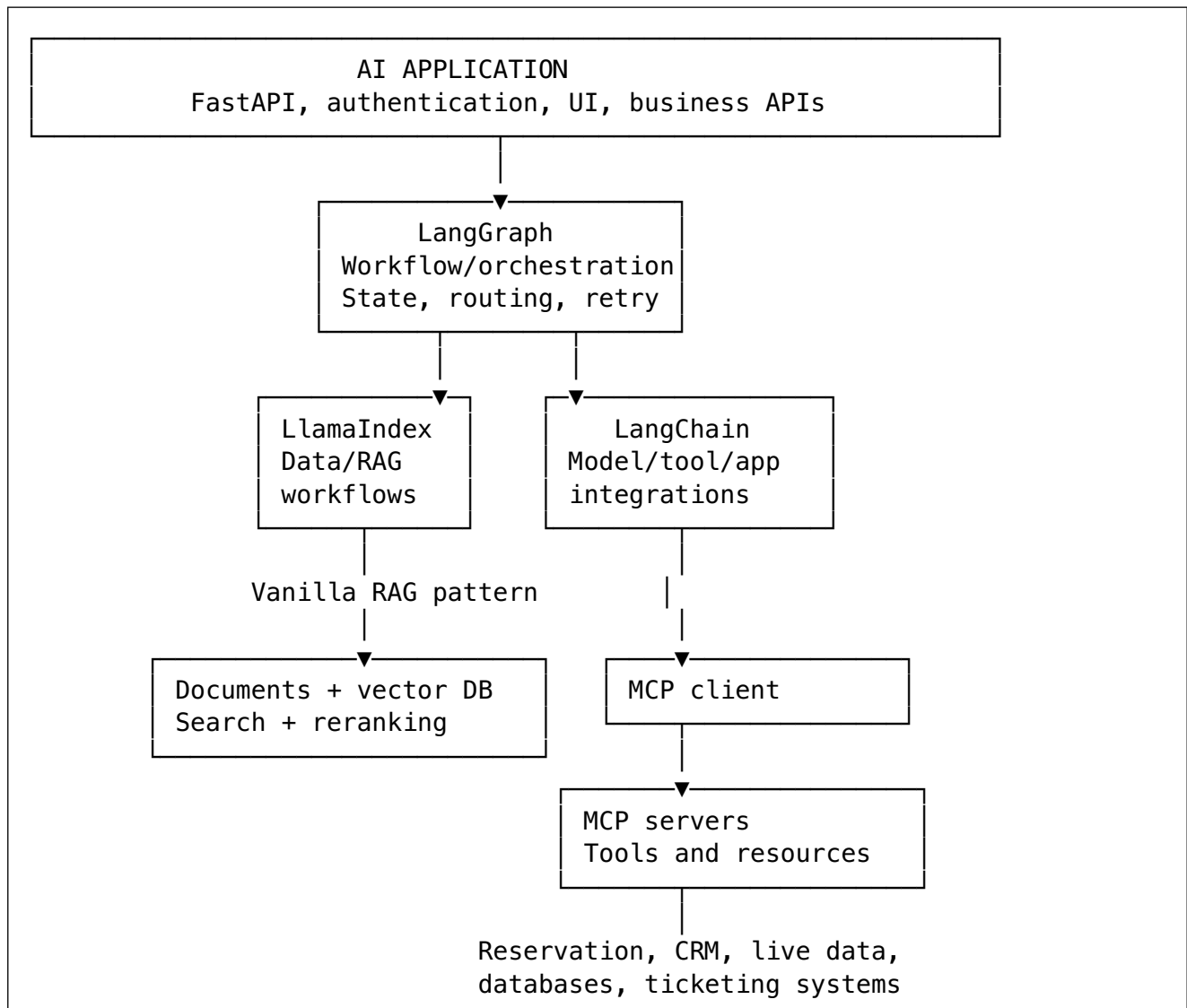
| “Can this framework do retrieval?”

Many frameworks can.

The better question is:

| “Which layer should own retrieval in our architecture, and which implementation gives us the best operational boundary?”

2.6 The category map



This diagram shows one possible combination, not a mandatory architecture.

3. Clear comparison table in plain English

Topic	Category	Main problem solved	Best at	Not mainly meant for
Vanilla RAG	Design pattern	Giving a model relevant external knowledge	Simple retrieval, grounding and document Q&A	Complex workflow state, standardized tool connectivity
LlamaIndex	Data and AI application framework	Preparing and using private or enterprise data with LLMs	Ingestion, parsing, nodes, indexes, retrieval, query engines, data-oriented agents and workflows	Being a universal enterprise protocol
LangChain	LLM/agent application framework	Connecting models, prompts, tools, retrieval and integrations	Application assembly, model interfaces, tool interfaces, agent components and provider integrations	Durable custom workflow execution by itself
LangGraph	Stateful orchestration runtime	Reliably running controllable multi-step AI workflows	State, branching, loops, persistence, retries, interrupts and human approval	Document parsing or data ingestion as its main purpose
MCP	Connectivity protocol	Standardizing connections between AI applications and external capabilities	Tool/resource discovery, standardized requests, server isolation and interoperability	Retrieval quality, model prompting or workflow decisions

LlamaIndex describes itself as a framework for building LLM-powered agents over data, including RAG pipelines, data connectors, indexes, retrievers, query interfaces, and workflows. ([Developer Documentation](#))

LangChain's current documentation emphasizes model/tool integrations and prebuilt agent architecture. It also provides retrieval components, document abstractions, vector-store integrations, and retrievers. ([Docs by LangChain](#))

LangGraph focuses on orchestration capabilities such as persistence, durable execution, state transitions, streaming, and human-in-the-loop operation. ([Docs by LangChain](#))

MCP servers expose capabilities through standardized tools, resources, and prompts. Tools perform actions, resources supply context, and prompts provide reusable interaction templates. ([Model Context Protocol](#))

What each is best at

Vanilla RAG

Use it when the problem is primarily:

| “Find relevant trusted information and answer from it.”

Example:

| “What is the cancellation policy for a character dining reservation?”

Vanilla RAG can be enough:

Question → retrieve cancellation-policy chunks → construct prompt → generate cited answer
--

You may not need an agent or graph.

LlamaIndex

Use it when your hardest problem is:

| “How do we turn complicated enterprise data into good model context?”

It is especially useful when you have:

- PDFs;
- policies;
- knowledge articles;
- tables;
- metadata;
- multiple indexes;
- advanced retrieval;
- document-specific parsing;
- several query engines.

LlamaIndex has strong abstractions around connecting, structuring, indexing, retrieving, and querying data. ([GitHub](#))

LangChain

Use it when your hardest problem is:

| “How do we assemble models, tools, prompts, retrieval and integrations into an application?”

It can provide standard interfaces for:

- model providers;
- embedding providers;
- tools;
- messages;
- output structures;
- retrievers;
- vector stores;

- agent loops.

LangGraph

Use it when your hardest problem is:

| “How do we control and reliably execute a stateful multi-step process?”

For example:

```
Classify request
→ retrieve policy
→ check live wait time
→ ask for confirmation
→ update reservation
→ verify result
→ respond
```

This workflow has:

- multiple paths;
- saved state;
- external side effects;
- approval;
- error recovery.

That is a strong LangGraph use case.

MCP

Use it when your hardest problem is:

| “How can many AI applications connect to many tools without writing a unique integration for every pairing?”

Without MCP:

```
MagicGuide → custom Reservation integration
MagicGuide → custom CRM integration
MagicGuide → custom Ticketing integration

Support Copilot → another Reservation integration
Support Copilot → another CRM integration
Support Copilot → another Ticketing integration
```

With MCP:

```
AI applications
  ↓ standard MCP interface
Reservation MCP server
CRM MCP server
Ticketing MCP server
```

MCP clients and servers negotiate capabilities and communicate through standardized operations rather than every application inventing its own tool contract. ([Model Context Protocol](#))

What each is not

Vanilla RAG is not

- a Python library;
- a workflow engine;
- an agent;
- a security protocol;
- a vector database.

LlamaIndex is not

- only a vector database wrapper;
- only a PDF loader;
- a mandatory part of RAG;
- an industry connectivity standard.

LangChain is not

- a model;
- a vector database;
- automatically a production architecture;
- a replacement for all business logic.

LangGraph is not

- simply a visual diagram library;
- only for fully autonomous agents;
- a retrieval engine;
- dependent on LangChain for every use case.

Officially, LangGraph can use LangChain components, but LangChain is not required to use LangGraph. ([LangChain AI](#))

MCP is not

- an agent framework;
- a reasoning engine;
- a replacement for REST, databases or internal services;
- a guarantee that a tool is safe;
- a place where the main conversation state should necessarily live.

MCP manages communication and connection lifecycle. Application-level workflow state normally belongs in the host application or orchestration layer.

Common misconceptions

Misconception 1: “LlamaIndex and LangChain are competitors, so choose only one.”

They overlap, but they can complement each other.

Example:

```
LlamaIndex:
    ingest and retrieve policy documents

LangChain:
    model wrapper, message handling and tool adapters

LangGraph:
    control the complete workflow
```

You can also choose only one framework when that keeps the system simpler.

Misconception 2: “LangGraph replaces LangChain.”

Not exactly.

A practical distinction is:

```
LangChain = components and higher-level agent abstractions
LangGraph = lower-level execution and orchestration runtime
```

LangChain’s current agent abstraction is built on LangGraph, while raw LangGraph provides more control. ([Docs by LangChain](#))

Misconception 3: “MCP replaces LangChain tools.”

MCP defines how remote or local capabilities are exposed and called.

LangChain may wrap an MCP capability as a LangChain-compatible tool.

```
MCP server
  ↓
MCP client/adaptor
  ↓
LangChain tool abstraction
  ↓
LangGraph node or agent
```

Misconception 4: “Using a framework automatically makes RAG production-ready.”

No.

Production readiness still requires:

- data quality;
- access control;
- evaluation;
- observability;
- deployment;
- caching;
- fallback;
- security testing;
- incident ownership.

Misconception 5: “Every AI request should use an agent.”

No.

A deterministic RAG flow is often:

- cheaper;
- faster;
- easier to test;
- easier to explain;
- safer.

Use agentic decisions only where they provide real value.

4. Inter-relation between all five topics

4.1 Vanilla RAG is the retrieval backbone

RAG answers:

| “What knowledge should the model see before answering?”

For the MagicGuide request:

| “Can my daughter ride Starlight Mountain?”

The RAG system retrieves:

- attraction eligibility policy;
- height restrictions;
- guardian rules;
- accessibility guidance;
- the policy version and effective date.

The basic path is:

```
Question
  ↓
Query understanding
  ↓
Search policy index
  ↓
Rerank passages
  ↓
Build grounded context
  ↓
Generate answer
```

RAG is the idea behind this path.

4.2 LlamaIndex implements the data-heavy part

LlamaIndex could own the offline ingestion process:

```
Policy source
  ↓
Document loader
  ↓
Parsing and cleaning
  ↓
Chunking into nodes
  ↓
Metadata enrichment
  ↓
Embedding generation
  ↓
Vector and keyword indexes
```

Metadata might include:

```
{
  "park": "Magic Kingdom-like park",
  "attraction": "Starlight Mountain",
  "document_type": "safety_policy",
  "effective_date": "2026-06-01",
  "language": "en",
  "region": "US",
  "access_level": "public"
}
```

At query time, LlamaIndex could provide:

- retrievers;
- metadata filters;
- query engines;
- reranking;
- response synthesizers;
- data-focused workflows.

The important boundary is:

| LlamaIndex owns how enterprise knowledge becomes useful context.

4.3 LangChain assembles application components

LangChain might provide:

- a common chat-model interface;
- prompt templates;
- structured output;
- tool definitions;
- retriever interfaces;
- model-provider switching;
- callback or tracing hooks;
- MCP adapters.

Example application components:

```
intent_classifier_model
policy_retriever
live_wait_time_tool
reservation_tool
answer_synthesis_prompt
structured_response_schema
```

The important boundary is:

| LangChain helps expose application capabilities through consistent interfaces.

You could write those interfaces yourself, but LangChain reduces repeated integration work.

4.4 LangGraph controls the request journey

The user's request is not one simple action.

It contains:

1. a knowledge question;
2. a live-data question;
3. a requested side effect.

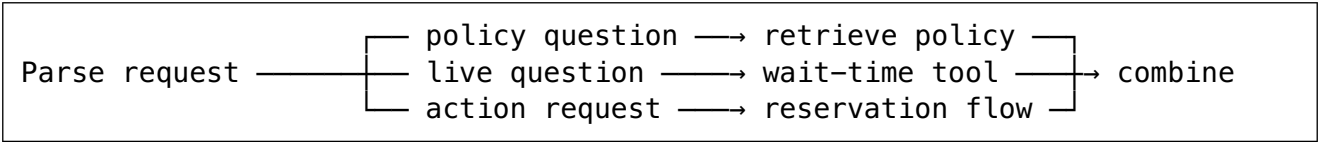
LangGraph can represent that as state:

```
state = {
    "user_request": "...",
    "guest_context": {},
    "intent": [],
    "retrieved_policy": [],
    "live_wait_time": None,
    "reservation": None,
    "user_approved_change": False,
    "tool_results": [],
    "errors": [],
    "final_answer": None
}
```

Possible nodes:

```
authenticate_user
parse_request
retrieve_policy
check_wait_time
fetch_reservation
request_confirmation
modify_reservation
verify_modification
generate_response
evaluate_response
```

Possible routing:



LangGraph’s shared state, nodes, and edges are built for exactly this kind of controlled multi-step execution. ([Docs by LangChain](#))

4.5 MCP connects external systems

The reservation platform may be maintained by another team.

Instead of adding its SDK directly inside MagicGuide, that team exposes an MCP server:

Reservation MCP server

Tools:

- get_reservation
- find_available_slots
- update_reservation
- cancel_reservation

Resources:

- reservation_schema
- modification_policy
- supported_restaurants

Prompts:

- assist_with_reservation_change

MCP has three especially useful server-side concepts:

MCP concept	Meaning in MagicGuide
Tool	Actively check or update a reservation
Resource	Read reservation schema or restaurant information
Prompt	Reusable workflow template for reservation assistance

Official MCP guidance distinguishes these by control: models generally request tools, applications choose resources for context, and users explicitly select prompts. ([Model Context Protocol](#))

The important boundary is:

MCP standardizes how the AI application discovers and communicates with the external capability.

It does not decide that the reservation should be modified. LangGraph or application business logic decides that.

4.6 Where model providers sit

Model providers sit below the model abstraction:

```
LangChain model interface
    ↓
Model gateway
    ↓
OpenAI / Anthropic / Google / AWS / Azure / internal model
```

The model is responsible for tasks such as:

- intent classification;
- extraction;
- reasoning;
- tool selection;
- response generation.

The model should not directly access databases.

Use controlled interfaces:

```
Model
  ↓ request
Application/orchestrator
  ↓ validated operation
Retriever or tool
```

4.7 Where vector databases sit

The vector database belongs in the retrieval/data layer:

```
LlamaIndex or custom retriever
    ↓
Vector database
```

The vector database stores and searches vectors. It does not:

- generate answers;
- control workflows;
- decide whether to call tools;
- enforce complete application governance.

You may also use:

- keyword search;
- relational databases;
- graph databases;
- search engines;
- reranking models.

RAG does not require vector-only search.

4.8 Where normal APIs sit

MCP does not remove ordinary APIs.

A realistic path is:

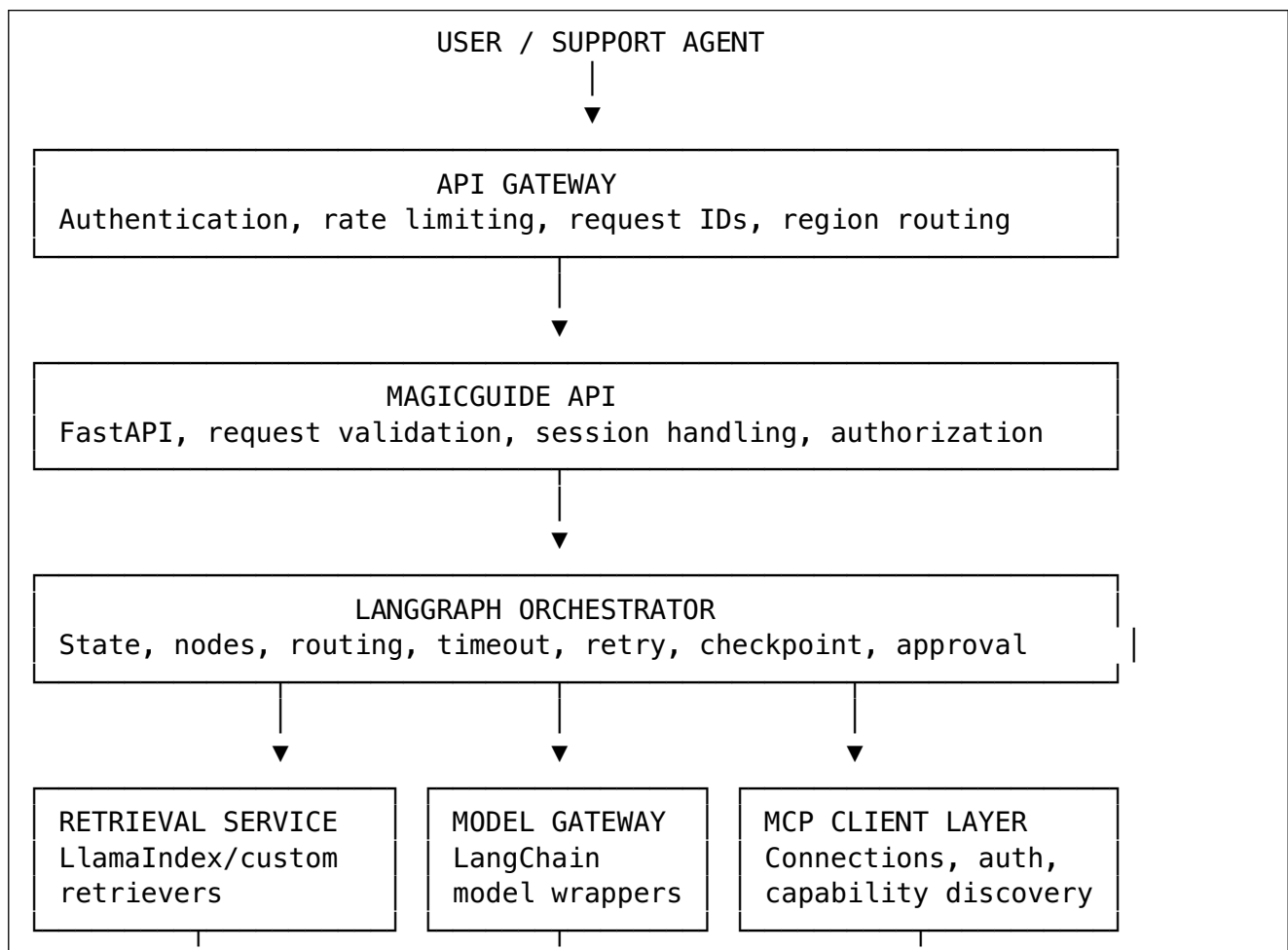
```
LangGraph
  ↓
MCP client
  ↓
Reservation MCP server
  ↓
Existing internal REST/gRPC reservation API
  ↓
Reservation database
```

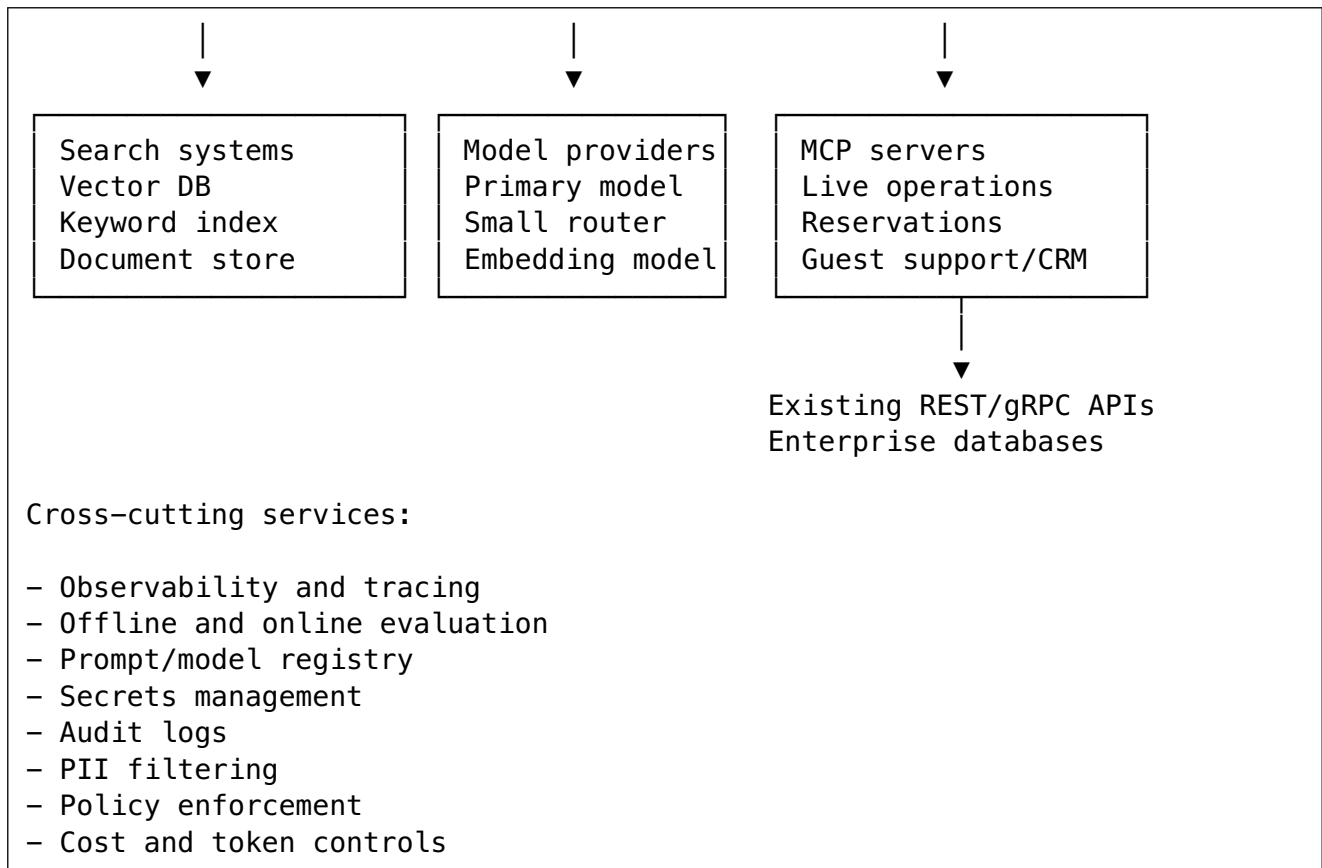
The MCP server acts as an AI-friendly adapter around existing enterprise services.

This allows the reservation team to keep its normal service architecture while providing a standardized interface for approved AI clients.

5. One end-to-end production architecture example

5.1 Architecture





5.2 Full request lifecycle

The user asks:

“Can my daughter ride Starlight Mountain, what is the current wait, and can you move our lunch reservation to 2:30 PM?”

Step 1: Request enters the API

The API gateway:

- authenticates the user;
- applies rate limits;
- creates a trace ID;
- sends the request to MagicGuide.

Example:

```
{
  "trace_id": "req-48291",
  "user_id": "guest-123",
  "session_id": "session-789",
  "message": "Can my daughter ride...",
  "locale": "en-US"
}
```

Step 2: Validate authorization

The backend checks:

- Is the user signed in?
- Can this user see the reservation?
- Is the request coming from an approved region?
- Is parental or guest consent required?
- Are any sensitive fields present?

The model does not make this authorization decision.

Use deterministic application code.

Step 3: Create graph state

LangGraph initializes the request state:

```
Request text
Authenticated identity
Conversation context
Permissions
Deadlines
Cost budget
Tool-call budget
```

LangGraph persistence can store checkpoints so a workflow can resume after an interruption or failure. ([Docs by LangChain](#))

Step 4: Understand the request

A small model or deterministic classifier identifies:

```
{
  "intents": [
    "attraction_eligibility",
    "live_wait_time",
    "modify_reservation"
  ],
  "entities": {
    "attraction": "Starlight Mountain",
    "requested_time": "14:30"
  }
}
```

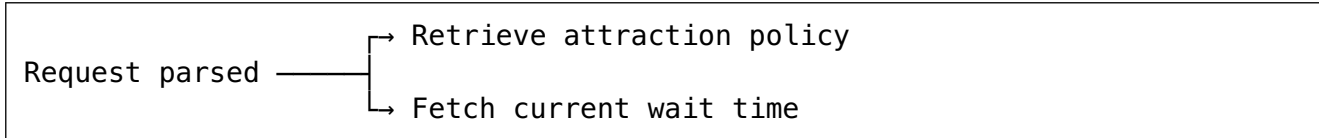
This output must follow a validated schema.

If parsing confidence is low:

```
route → clarification node
```

Step 5: Run independent read operations in parallel

The policy retrieval and live wait-time lookup do not depend on each other, so they may run concurrently.



Parallel execution reduces latency.

Step 6: Retrieval path

The retrieval node calls the LlamaIndex-based retrieval service.

The service:

1. rewrites the query if needed;
2. applies park and attraction metadata filters;
3. performs hybrid search;
4. retrieves candidate chunks;
5. reranks them;
6. removes outdated or unauthorized documents;
7. returns passages with citations.

Example result:

```
{
  "passages": [
    {
      "document_id": "policy-921",
      "section": "Eligibility",
      "text": "...",
      "effective_date": "2026-06-01",
      "score": 0.92
    }
  ]
}
```

The retrieval service should return evidence, not only a generated answer. This allows the orchestration and evaluation layers to inspect what was retrieved.

Step 7: Live tool path

The wait-time node calls the Live Operations MCP server:

```
tools/list
  ↓
get_attraction_wait_time(
  attraction_id="starlight-mountain"
)
```

The MCP client validates the tool schema and sends the request to the correct server.

MCP's standardized architecture allows the host application to maintain separate client connections to different servers, creating clearer isolation and permission boundaries. ([Model Context Protocol](#))

Step 8: Reservation read path

Before modifying anything, the workflow retrieves the current reservation:

```
get_reservation(reservation_id)
```

It then checks available times:

```
find_available_slots(  
    restaurant_id,  
    requested_date,  
    party_size  
)
```

Read and write tools should be separate.

This is safer than one broad tool such as:

```
manage_reservation(anything)
```

Step 9: Policy and business-rule check

Deterministic code verifies:

- Does the reservation belong to the user?
- Is 2:30 PM available?
- Is the change within the allowed modification window?
- Will a fee apply?
- Is the restaurant or party size compatible?
- Does the user have permission to modify it?

The LLM may explain the rule, but application code should enforce it.

Step 10: Human confirmation

Changing a reservation is a side effect.

The graph pauses and asks:

```
“A 2:30 PM slot is available. Moving the reservation may replace your current 1:00 PM slot.  
Shall I proceed?”
```

LangGraph interrupts can pause execution, save state, and resume after external user input, which supports human-in-the-loop workflows. ([Docs by LangChain](#))

Step 11: Execute the write tool

After confirmation:

```
update_reservation(  
    reservation_id="R-123",  
    new_time="14:30",  
    idempotency_key="req-48291-change-1"  
)
```

The idempotency key prevents accidental duplicate changes if the request is retried.

Step 12: Verify the change

Do not trust only a successful tool response.

Call:

```
get_reservation(reservation_id)
```

Then verify that the time is really 2:30 PM.

This is a Staff-level reliability pattern:

| Execute, then verify.

Step 13: Response synthesis

The final model receives:

- retrieved policy evidence;
- current wait time;
- verified reservation result;
- response rules;
- citation requirements.

It creates a response such as:

Based on the current attraction policy, your daughter meets the stated eligibility requirement. The current estimated wait is 45 minutes.

Your lunch reservation was moved from 1:00 PM to 2:30 PM.
Confirmation: R-123.

Source: Attraction Eligibility Policy, effective June 1, 2026.

The model should not claim an action succeeded unless the verified tool result confirms it.

Step 14: Output validation

Before returning the response, validate:

- Does every policy claim have evidence?
- Did the response expose PII?
- Does the reservation status match the tool result?
- Did the model invent any confirmation number?
- Is the answer in the requested language?
- Is uncertainty communicated correctly?

Step 15: Logging and tracing

Record:

Trace ID
Graph path
Node latency
Model and prompt version
Retrieved document IDs
Retrieval scores
Tool name and result status
User approval event
Token usage
Estimated cost
Final evaluation result

Sensitive tool arguments and personal data should be redacted or tokenized.

5.3 Simplified workflow pseudocode

```
def handle_request(request):
    state = authenticate_and_initialize(request)

    intents = classify_request(state)

    if "attraction_eligibility" in intents:
        state.policy_evidence = retrieve_policy(state)

    if "live_wait_time" in intents:
        state.wait_time = call_live_operations_mcp(state)

    if "modify_reservation" in intents:
        state.reservation = fetch_reservation_mcp(state)
        state.available_slots = find_slots_mcp(state)

        validate_business_rules(state)

        if requires_confirmation(state):
            pause_for_user_confirmation(state)

        state.update_result = update_reservation_mcp(state)
        state.verified_reservation = verify_reservation_mcp(state)

    state.final_answer = generate_grounded_response(state)
    validate_final_answer(state)

    return state.final_answer
```

Notice the separation:

```
Model suggests or extracts  
Code validates  
Graph controls  
RAG supplies knowledge  
MCP connects  
External services execute
```

6. Production-grade challenges across the full stack

6.1 Too much framework complexity

A common mistake is:

```
LlamaIndex agents  
inside LangChain agents  
inside LangGraph  
calling MCP  
through another internal abstraction
```

This creates:

- too many data types;
- duplicated retries;
- duplicated memory;
- difficult traces;
- unclear ownership;
- dependency conflicts.

Staff-level response

For every layer, ask:

| “What unique responsibility does this layer own?”

Example:

```
LlamaIndex → retrieval only  
LangChain → normalized model and tool interfaces  
LangGraph → workflow only  
MCP → external connectivity only
```

6.2 Wrong tool choice

Teams sometimes use an LLM for deterministic logic.

Bad example:

| “Ask the model whether the user owns this reservation.”

Good example:


```
reservation.user_id == authenticated_user.id
```

Use models for:

- language understanding;
- extraction;
- synthesis;
- uncertain classification.

Use code for:

- authorization;
 - money;
 - eligibility enforcement;
 - state transitions;
 - limits;
 - irreversible actions.
-

6.3 Poor boundaries between layers

A retrieval node should not secretly modify a reservation.

A model wrapper should not contain business authorization.

An MCP server should not control the entire conversation.

Poor boundaries make the system difficult to test.

A cleaner contract is:

```
Retriever:
query → evidence

Model:
messages/context → structured output

Tool:
validated arguments → result

Graph node:
state → state update

MCP client:
standard request → standard response
```

6.4 Debugging difficulty

When an answer is wrong, the cause could be:

- bad source document;
- poor chunking;
- weak retrieval query;

- vector-search miss;
- incorrect reranking;
- prompt error;
- model reasoning error;
- wrong graph route;
- MCP connection failure;
- tool implementation bug;
- stale external data.

Without per-layer traces, everything looks like:

| “The AI gave a wrong answer.”

That is not actionable.

You need to see:

```
What did the user ask?
What intent was detected?
What documents were retrieved?
What tools were available?
What tool was selected?
What arguments were sent?
What result returned?
Which graph path executed?
What context reached the final model?
```

6.5 Security and governance risks

Major risks include:

- prompt injection inside retrieved documents;
- malicious tool descriptions;
- excessive tool permissions;
- user A accessing user B’s data;
- secrets appearing in prompts;
- unapproved external MCP servers;
- tool results containing malicious instructions;
- data crossing regional boundaries;
- irreversible actions without confirmation.

MCP standardizes communication, but the host still needs strong security controls. MCP’s authorization specification covers transport-level authorization for protected HTTP servers and emphasizes audience-bound access tokens rather than passing arbitrary downstream tokens through servers. ([Model Context Protocol](#))

Important controls include:

Allowlisted MCP servers Per-tool permissions Read/write separation Least-privilege tokens User confirmation Argument validation Output sanitization Network isolation Audit logging Data classification
--

Treat model output, retrieved content, and tool output as untrusted input.

6.6 High latency

The request might involve:

- routing model: 300 ms;
- retrieval: 600 ms;
- reranking: 400 ms;
- live tool: 800 ms;
- reservation lookup: 700 ms;
- confirmation;
- final model: 1,500 ms.

Sequentially, this becomes slow.

Solutions include:

- parallel independent nodes;
 - smaller routing models;
 - caching safe reads;
 - connection pooling;
 - fewer model calls;
 - retrieval before generation;
 - timeouts and fallbacks;
 - streaming partial responses;
 - precomputed embeddings;
 - regional deployment.
-

6.7 High cost

Costs may come from:

- embedding generation;
- vector storage;
- reranking;
- multiple LLM calls;
- large retrieved contexts;
- repeated agent loops;

- unused tool calls;
- verbose conversation history.

Common cost controls:

Per-request token budget Per-graph model-call limit Per-agent tool-call limit Context compression Semantic caching Small models for routing Large models only for difficult synthesis Offline document deduplication

6.8 Weak observability

Basic API logs are not enough.

You need AI-specific visibility:

Retrieval metrics

- retrieval latency;
- top-k results;
- recall;
- precision;
- reranker changes;
- empty retrieval rate;
- stale-document rate.

Model metrics

- token usage;
- time to first token;
- completion latency;
- structured-output failures;
- provider error rate;
- refusal rate.

Workflow metrics

- node execution count;
- graph path;
- retry count;
- interruption count;
- abandonment rate;
- workflow completion rate.

Tool metrics

- tool-selection accuracy;
- argument-validation failures;
- tool latency;

- tool success rate;
- side-effect verification failure.

6.9 Weak evaluation

A single “answer quality” score is insufficient.

Evaluate each layer independently.

Layer	Example metrics
Retrieval	Context recall, precision, relevance
Generation	Groundedness, correctness, citation coverage
Routing	Intent classification accuracy
Tool use	Correct tool, correct arguments, successful result
Workflow	Completion rate, recovery rate
Safety	Unauthorized-action rate, PII leakage
Operations	p95 latency, error rate, cost per successful request

Also run end-to-end scenarios:

Correct policy + successful update
 Correct policy + no available slot
 Retrieval failure
 MCP timeout
 Tool returns partial success
 User denies confirmation
 Duplicate update request
 Outdated policy document
 Prompt-injected document
 Unauthorized reservation access

6.10 Provider lock-in

Lock-in can happen through:

- model-specific message formats;
- provider-specific tool schemas;
- proprietary vector filters;
- framework-specific document objects;
- framework-managed state formats;
- provider-specific tracing.

Use internal interfaces:

```
class ModelGateway:
    def generate(self, request): ...

class KnowledgeRetriever:
    def retrieve(self, query, filters): ...

class ReservationService:
    def update(self, command): ...
```

Framework objects should normally stay behind adapters.

Do not allow every business service to depend directly on LangChain, LlamaIndex, or MCP-specific objects.

6.11 Operational ownership confusion

When something fails, who owns it?

Possible teams:

```
Document ingestion team
Search/retrieval team
AI application team
Model platform team
Workflow platform team
MCP platform team
Reservation service team
Security and governance team
```

Define ownership explicitly.

Example:

Component	Owner
Policy sources and freshness	Content operations
Parsing and indexing	Knowledge platform
Retrieval service	Search/AI platform
Graph workflow	MagicGuide application team
MCP client platform	AI platform
Reservation MCP server	Reservation domain team
Underlying reservation API	Reservation backend team
Evaluation suite	AI quality team with product owners

A Staff Engineer designs not only the software boundary but also the ownership boundary.

7. Optimization strategies across the full stack

7.1 Start with the simplest architecture

Use this maturity path:

Level 1: Plain prompt

User → model

Use when no private or current knowledge is needed.

Level 2: Simple RAG

User → retrieve → model

Use for document Q&A.

Level 3: RAG plus deterministic tools

User → route → retrieval/tool → model

Use when a small number of predictable actions exist.

Level 4: Stateful graph

User → graph with branches, retry and approval

Use when workflows become genuinely multi-step.

Level 5: Standardized MCP connectivity

Multiple AI apps → shared MCP servers → enterprise systems

Use when tool reuse and interoperability justify the protocol layer.

Do not begin at Level 5 merely because it looks advanced.

7.2 Use clear separation of concerns

A strong separation is:

API layer: Authentication, request validation, rate limiting
Orchestration layer: State, routing, retry, approval, recovery
Knowledge layer: Ingestion, indexing, retrieval, reranking
Application component layer: Model, prompt and tool interfaces
Connectivity layer: MCP clients and servers
Domain services: Reservations, ticketing, CRM, park operations
Cross-cutting layer: Security, observability, evaluation and cost

7.3 Use simple RAG when enough

Choose plain RAG when:

- the request is read-only;
- one knowledge source is enough;
- one retrieval call is enough;
- there are no complex decisions;
- there are no side effects;
- failures can return a simple fallback.

Example:

| “Summarize the stroller policy.”

You may need only:

FastAPI + retriever + model

No LangGraph. No MCP. Possibly no LlamaIndex or LangChain.

7.4 Use LlamaIndex where data pipelines matter

Choose LlamaIndex when you need strong support for:

- many data formats;
- complex documents;
- data connectors;
- document parsing;
- metadata;

- multiple indexes;
- hierarchical retrieval;
- query engines;
- data-oriented workflows.

The decision should come from data complexity, not popularity.

7.5 Use LangChain where application composition matters

Choose LangChain when you benefit from:

- normalized model APIs;
- reusable tool abstractions;
- structured output;
- provider integrations;
- retriever interfaces;
- agent middleware or higher-level agent patterns.

It is especially useful when you expect providers or integrations to change.

However, avoid wrapping five lines of simple code in many abstractions without a real benefit.

7.6 Use LangGraph where workflow control matters

Use LangGraph when you need several of these:

- branching;
- loops;
- persistent state;
- recovery;
- retries;
- long-running work;
- parallel steps;
- human approval;
- deterministic and agentic steps together;
- replay and inspection.

Do not use it only because your application has two sequential function calls.

7.7 Use MCP where standardized connectivity matters

MCP is valuable when:

- several AI applications need the same capabilities;
- tools are maintained by different domain teams;
- capability discovery matters;
- local and remote servers need a common contract;
- you want clearer isolation between AI hosts and integrations;
- standardized resources, tools, and prompts create reuse.

Use direct APIs when:

- one application calls one stable service;
 - the extra protocol layer provides little benefit;
 - strict latency requirements make another hop undesirable;
 - your platform is not yet ready to govern MCP servers.
-

7.8 Design good interfaces

Retriever interface

```
retrieve(  
    query,  
    tenant_id,  
    access_scope,  
    metadata_filters,  
    top_k  
) -> EvidenceBundle
```

Tool interface

```
execute(  
    authenticated_principal,  
    validated_arguments,  
    idempotency_key  
) -> ToolResult
```

Graph node interface

```
node(state: WorkflowState) -> StateUpdate
```

Model interface

```
generate(  
    messages,  
    response_schema,  
    budget,  
    safety_policy  
) -> ModelResult
```

Interfaces should use your domain objects, not expose framework internals everywhere.

7.9 Use reliability patterns

For external operations:

```
Timeout
Retry only safe operations
Circuit breaker
Idempotency
Bulkhead isolation
Fallback
Execute then verify
Dead-letter handling
```

Be careful with retries.

Safe retry:

```
get_wait_time
retrieve_policy
```

Potentially dangerous retry:

```
charge_payment
cancel_booking
update_reservation
```

Write operations require idempotency and verification.

7.10 Keep controls across every layer

Security, evaluation, observability and cost are not separate final steps.

They must exist throughout:

```

                Security
                ↓
API → Graph → Retrieval → Model → MCP → Service
                ↑
        Tracing + evaluation + cost
```

For example:

- API checks identity.
- Graph checks action approval.
- Retriever applies document permissions.
- Model gateway applies token budgets.
- MCP layer restricts servers and tools.
- Domain service rechecks authorization.
- Audit system records the outcome.

This is defence in depth.

8. Staff-level interview angle

8.1 How to explain the full stack

A good interview answer:

“I separate the system into architectural responsibilities. RAG is the knowledge-grounding pattern. I may use LlamaIndex for ingestion, indexing and advanced retrieval over enterprise content. LangChain can normalize model, prompt, retriever and tool integrations. LangGraph controls the stateful request lifecycle, including routing, retry, persistence and human approval. MCP standardizes connectivity to externally owned tools and resources. The model provider, vector database and business APIs remain independent infrastructure components behind these layers.”

That answer demonstrates categorization rather than framework memorization.

8.2 How to choose the right combination

Ask these questions in order.

Question 1: Does the system need external knowledge?

If no:

Model or normal backend logic

If yes:

Use retrieval/RAG

Question 2: How difficult is the data pipeline?

Simple:

Custom ingestion and retriever

Complex:

Consider LlamaIndex

Question 3: Do we need reusable model and tool integrations?

If yes:

Consider LangChain or an internal equivalent

Question 4: Is the workflow stateful and multi-step?

If yes:

Consider LangGraph

Question 5: Do multiple AI applications need standardized access to external systems?

If yes:

Consider MCP

Question 6: Is the complexity justified?

Always ask:

Can we solve this safely with fewer layers?

8.3 How to justify trade-offs

Do not say:

| “We used LangGraph because it is good for agents.”

Say:

| “The workflow requires parallel policy retrieval and availability checks, a persistent confirmation step before a reservation mutation, retry-safe execution, and recovery after external API failures. A stateful graph makes these transitions explicit and testable.”

Do not say:

| “We used LlamaIndex because it is best for RAG.”

Say:

| “Our knowledge base contains complex policy PDFs, versioned metadata, multilingual content and several retrieval strategies. LlamaIndex reduces custom ingestion and retrieval code while allowing us to preserve a clean retrieval-service contract.”

Do not say:

| “We used MCP because MCP is the future.”

Say:

| “Reservation, CRM and park-operation capabilities are consumed by several assistants and maintained by different domain teams. MCP gives us one discoverable tool contract and allows each domain team to operate its server independently. For one private API used by only one application, I would probably use the API directly.”

8.4 How to answer “Why this framework and not that one?”

Use this four-part structure:

1. State the requirement

| “We need persistent workflow state and human approval.”

2. State the chosen tool’s strength

| “LangGraph gives us explicit nodes, conditional edges, checkpointing and interrupts.”

3. Explain why alternatives are less suitable for this responsibility

“LlamaIndex workflows can also coordinate data-oriented tasks, but our primary need is application-wide orchestration across retrieval and transactional tools.”

4. State the cost

“The cost is another runtime abstraction and operational dependency, so we would use it only after a simpler deterministic service becomes difficult to maintain.”

This shows balanced engineering judgment.

8.5 Disney-like system-design answer

Imagine the interviewer asks:

“Design an AI guest-support platform for Disney parks.”

A Staff-level answer could be:

Business requirements

- Answer park-policy questions.
- Provide live attraction and reservation information.
- Support approved reservation changes.
- Serve multiple channels.
- Provide citations.
- Protect guest information.
- Support multiple regions and languages.
- Maintain high availability.

Architecture choices

Vanilla RAG:

Ground answers in approved park and guest-service documents.

LlamaIndex:

Parse, version, index and retrieve policies and operational documents.

LangChain:

Normalize model access, structured outputs and tool integrations.

LangGraph:

Control multi-intent requests, parallel reads, approval, writes, verification and failure recovery.

MCP:

Expose reservation, park operations and support capabilities through standardized domain-owned servers.

Reliability

- deterministic authorization;
- read/write tool separation;
- idempotent mutations;

- execute-and-verify;
- retrieval and model fallbacks;
- graph checkpointing;
- circuit breakers;
- regional failover.

Safety

- least privilege;
- tenant and guest-level filtering;
- sensitive-action confirmation;
- allowlisted MCP servers;
- prompt-injection defences;
- redacted logs;
- full action auditing.

Evaluation

- policy-answer correctness;
- retrieval recall;
- citation coverage;
- routing accuracy;
- tool-selection accuracy;
- reservation completion rate;
- unauthorized-action rate;
- p95 latency;
- cost per successful resolution.

Organizational design

- domain teams own MCP servers and underlying APIs;
- knowledge platform owns ingestion and indexes;
- AI platform owns model gateway and common runtime;
- product teams own graph workflows and guest experience;
- security owns cross-layer policy and auditing.

That is the difference between a framework-level answer and a Staff Engineer answer.

Final mental model

Remember this sentence:

RAG supplies knowledge, LlamaIndex organizes and retrieves data, LangChain assembles AI components, LangGraph controls the journey, and MCP connects the journey to external systems.

Or even more simply:

Vanilla RAG = What knowledge should we retrieve?

LlamaIndex = How do we prepare and retrieve enterprise knowledge?

LangChain = How do we connect models, prompts and tools?

LangGraph = What steps run, in what order, with what state?

MCP = How do we communicate with external tools and data consistently?

The most important Staff-level lesson is:

Do not use all five because you can. Use each only when its architectural responsibility is real, clear and operationally owned.